# PRACTICAL 9-B

**Object Oriented Programming**

## AIM:

To demonstrate race conditions and make shared account operations thread-safe using threads and synchronization.

## CODE SCREENSHOT:



```java
class Account {
    private long balance;

    synchronized void deposit(long amount) {
        balance += amount;
    }

    synchronized boolean withdraw(long amount) {
        if (amount <= 0 || amount > balance)
            return false;
        balance -= amount;
        return true;
    }

    synchronized long getBalance() {
        return balance;
    }
}

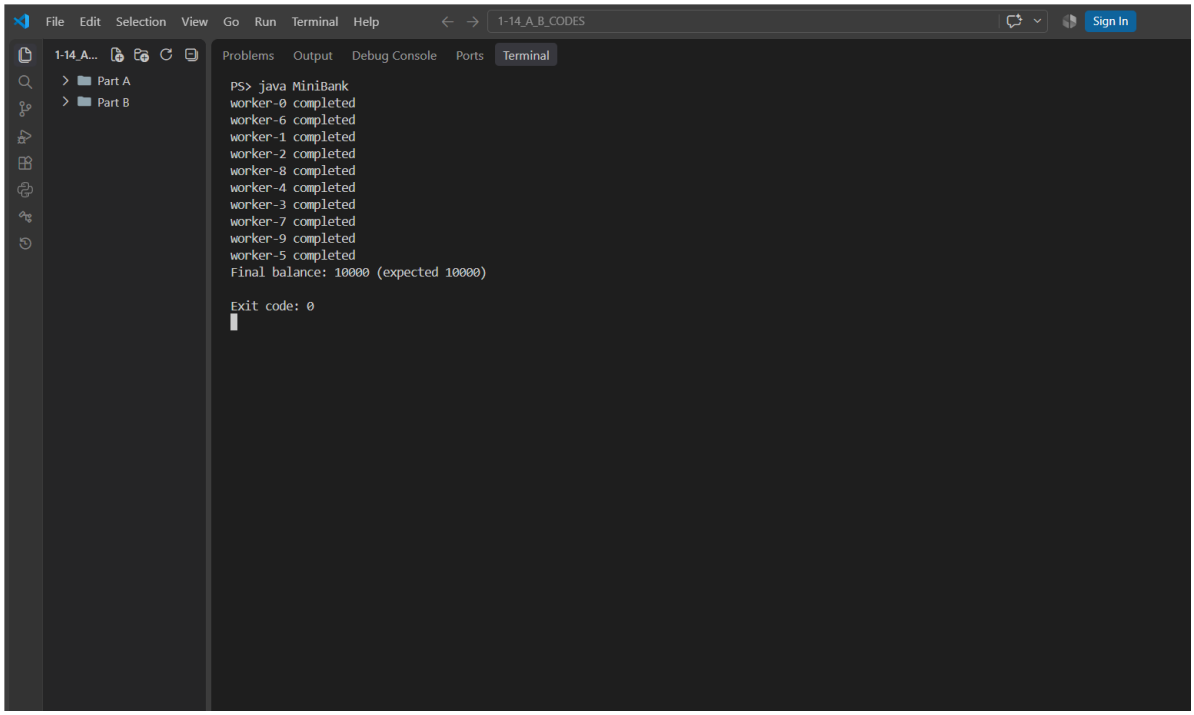
class AccountWorker implements Runnable {
    private final Account account;
    private final int times;
    private final long amount;

    AccountWorker(Account a, int t, long m) {
        account = a;
        times = t;
        amount = m;
    }

    public void run() {
        for (int i = 0; i < times; i++)
            account.deposit(amount);
        System.out.println(Thread.currentThread().getName() + " completed");
    }
}

public class MiniBank {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Account a = new Account();
        Thread[] workers = new Thread[10];
        for (int i = 0; i < workers.length; i++) {
            workers[i] = new Thread(new AccountWorker(a, t: 100, m: 10), "worker-" + i);
            workers[i].start();
        }
        for (Thread t : workers)
            t.join();
```

## OUTPUT SCREENSHOT:




## LEARNING OUTCOMES:

The student can create threads and protect shared data using synchronization.